

How It Works

System Explainer + Scenario Q/A

Iqra Digital Library v2 | LangGraph ReAct Orchestrator Architecture | May 2026

What this document covers

Part 1 ? three plain-English paragraphs explaining the entire system from user input to final response. Part 2 ? six scenario-based questions with detailed answers covering debugging, concurrency, memory, feature additions, and production failures.

Contents

- Part 1** How It Works ? 3 Paragraphs
- Part 2** Scenario 1 ? Agent gives completely wrong answer
 Scenario 2 ? 'Save the book you just found' fails
 Scenario 3 ? Ollama shows connected but agent gives no response
 Scenario 4 ? Client wants memory to survive server restarts
 Scenario 5 ? Two users send messages at the same time
 Scenario 6 ? Add PDF export feature with minimal code change

Part 1 ? How It Works

Paragraph 1 ? The Orchestrator is the Brain

Every time a user types a message in the AI Librarian tab, `app.py` calls `agent_mgr.chat(message, thread_id)`. This sends the message into the Orchestrator's LangGraph graph as a `HumanMessage`. The Orchestrator's agent node passes the full conversation history to the LLM, which reads the system prompt and decides what to do. It has exactly two tools: `search_worker` and `curator_worker`. If the user asked about books, it calls `search_worker`. If the user asked about their reading list, it calls `curator_worker`. If the request involves both ? find a book and save it ? it calls both in sequence. Once the tools return, the LLM reads the results and writes a final `AIMessage` with no `tool_calls`. That is the signal to exit the graph and return the reply to the user. `MemorySaver` records every message under the session's `thread_id`, so on the next turn the LLM remembers everything said before.

Paragraph 2 ? Each Worker is a Specialist with its Own Loop

When the Orchestrator calls `search_worker('mystery books set in Cairo')`, it is not just calling a function ? it is triggering a full independent ReAct graph inside `SearchWorker`. That graph's agent node looks at the query and picks from four tools: `search_local_library` (BM25 + FAISS + Knowledge Graph), `search_google_books` (live API), `find_similar_books` (semantic similarity), or `search_by_rating` (rating filter). It calls one, gets a `ToolMessage` back, and either calls another tool or writes its final answer. That final answer string is returned to the Orchestrator's tools node as a `ToolMessage`. The Curator Worker does the same thing independently for reading list operations ? `save_book`, `remove_book`, `view_reading_list`, `count_reading_list` ? reading and writing `data/reading_list.json` via the `ReadingList` class. Neither worker holds memory. They are stateless per call ? every invocation starts from a clean slate.

Paragraph 3 ? The Message List is the Shared Truth

Everything ? decisions, tool calls, tool results, final answers ? lives as typed message objects in the `MessagesState` list. `HumanMessage` carries the user input. `AIMessage` with `tool_calls` carries the LLM's decision to act. `ToolMessage` carries what the tool returned. `AIMessage` without `tool_calls` is the final answer. The graph routing is purely mechanical: if the last message has `tool_calls`, go to the tools node; otherwise go to `__end__`. This cycle repeats until the LLM stops calling tools. The `MemorySaver` checkpointer snapshots this list after every node so the next user message appends to a full history, giving the agent natural multi-turn memory without any manual state management from the caller.

Quick Reference ? Message Types

Message Type	Created by	Contains / Purpose
<code>HumanMessage</code>	User	Raw user text. Added before <code>__start__</code> on each <code>chat()</code> call.
<code>AIMessage (tool_calls)</code>	agent LLM	Decision to call a tool. Contains tool name + arguments as JSON.
<code>ToolMessage</code>	tools node	Result returned by a tool function. One per <code>tool_call</code> .
<code>AIMessage (no tool_calls)</code>	agent LLM	Final natural-language reply. Triggers exit to <code>__end__</code> .
<code>SystemMessage</code>	Graph init	System prompt at position 0 of every agent call. Not checkpointed.

Part 2 ? Scenario-Based Questions & Answers

Scenario 1: Agent gives completely wrong answer

User searched for "romance novels" and received programming books instead.

Q: How do you debug this?

A:

Pull the execution trace. The failure lives in exactly one of four places in the chain:

- Orchestrator routed wrong ? check whether it called `search_worker('romance novels')` or accidentally called `curator_worker`. If it called the wrong tool, the system prompt routing rules need to be more explicit.
- Search Worker picked the wrong tool ? check if it called `search_local_library('romance novels')` or received a corrupted query. If the query was mangled, the Orchestrator's tool call schema sent bad input.
- Retrieval returned wrong results ? the tool was called correctly but FAISS returned programming books for a romance query. The embedding space is poorly calibrated for this query type. Fix: add BM25 hybrid (catches the keyword 'romance') or add HyDE (generate a hypothetical romance novel description and embed that instead of the raw query).
- LLM ignored retrieved results ? tools returned correct books but the LLM hallucinated from its training weights. Fix: strengthen the system prompt with 'Only use the results from tools. Do not add books from memory.'

Add this exact

Scenario 2: 'Save the book you just found' fails

User says: "Save the first book you just found to my reading list." Agent replies: "I don't see any recent search results."

Q: What went wrong and how do you fix it?

A:

This is a memory scope bug. The user is referring to a result from a previous turn. The Orchestrator has MemorySaver so it should have the prior turn in state. Two likely causes:

- Different `thread_id`: the session was refreshed or reconnected, generating a new UUID. The new session has no history. Fix: persist `thread_id` to browser session storage so reconnects resume the same thread.
- Worker was asked without a concrete title: the Orchestrator called `curator_worker('save the first book from earlier')` but the `CuratorWorker` is stateless ? it has no memory of 'earlier'. Fix: the Orchestrator must extract the book title from its own message history BEFORE calling `curator_worker`, then call `curator_worker('save Dune by Frank Herbert')` with the concrete title.

Correct Orche
argument.

Scenario 3: Ollama shows connected but AI Librarian gives no response

Q: Walk through exactly what you check, step by step.

A:

Check four things in order:

- Step 1 ? agent_mgr.is_ready: if False, the agent was never built. This happens when LLM_PROVIDER / LLM_MODEL env vars trigger auto-connect at startup but agent_mgr.build() was not called after llm.configure(). Fix: ensure the auto-connect block calls agent_mgr.build() after a successful configure().
- Step 2 ? Model supports tool calling: if the user selected gemma:2b or phi3, those models do not support function/tool calling. The ReAct loop requires the LLM to output structured tool_calls JSON. Without it the agent errors or loops silently. Fix: only expose models that support tool calling: llama3.2, qwen2.5, mistral, llama3.1, deepseek-r1:8b.
- Step 3 ? Model is actually pulled: Ollama returns connected even if the specific model is not downloaded. The connection check must query /api/tags and verify the model name is in the pulled list. If not, show: 'Model not pulled yet. Run: ollama pull llama3.2'.
- Step 4 ? Ollama timeout: if the model is loading into VRAM, the initial request can take longer than 10 seconds. Raise the connection timeout to 30 seconds and add a user-visible loading indicator.

Scenario 4: Client wants memory to survive server restarts

A German enterprise client says: 'We want the AI to remember what employees searched last week, even if the server was restarted over the weekend.'

Q: How do you change the architecture to support this?

A:

MemorySaver is in-process RAM ? it dies on restart. Replace it with SqliteSaver or PostgresSaver. No other code changes are needed.

```
# SqliteSaver ? file-based, survives restarts, zero infrastructure
from langgraph.checkpoint.sqlite import SqliteSaver
memory = SqliteSaver.from_conn_string('checkpoints.db')

# PostgresSaver ? production-grade, multi-instance safe
from langgraph.checkpoint.postgres import PostgresSaver
memory = PostgresSaver.from_conn_string(os.getenv('DATABASE_URL'))

# Pass into the Orchestrator ? everything else stays identical
self._agent = create_react_agent(
    lc_llm, tools, checkpointinter=memory, prompt=_ORCHESTRATOR_SYSTEM_PROMPT
)
```

The thread_id key still works the same way. The chat() method and all tool logic are completely unchanged. For enterprise, also store the thread_id -> user_id mapping in a users table so employees see only their own history and cannot access another person's checkpointed conversations.

Scenario 5: Two users send messages at exactly the same time

Q: Does the system break? How does concurrency work?

A:

It is safe because MemorySaver isolates by thread_id. Each Gradio session generates a unique UUID as its thread_id. When User A's request arrives simultaneously with User B's, they are two separate invoke() calls with different thread_ids ? they access completely different checkpoints and never touch each other's state.

The Workers (SearchWorker, CuratorWorker) are stateless ? each call is independent, so two simultaneous searches run in parallel without conflict.

The only shared mutable state is data/reading_list.json. If two users trigger save_book at the exact same millisecond, there is a race condition on the file write. Fix for production: replace the JSON file with SQLite (WAL mode) or PostgreSQL with row-level locking. For single-user Gradio deployments this is not an issue ? Gradio's event queue serialises writes.

- Orchestrator state: isolated per thread_id ? safe
- Worker state: none ? safe
- reading_list.json: shared file ? race condition risk in multi-user deployments
- FAISS index: read-only at query time ? safe
- LLM API calls: independent per request ? safe

Scenario 6: Add PDF export with minimal code change

Product team request: "The agent should be able to export the reading list as a PDF when the user asks for it."

Q: What is the minimal change to the codebase?

A:

Add one tool to CuratorWorker only. Zero changes to the Orchestrator or SearchWorker. The exporter already exists at exporter.py ? just expose it as a new tool.

```
# In agents/curator_agent.py ? inside _build_tools()

@tool
def export_reading_list_pdf(placeholder: str = '') -> str:
    'Export the reading list to a PDF file and return the file path.'
    from exporter import export_books_pdf
    books = rl.get_all()
    if not books:
        return 'Reading list is empty ? nothing to export.'
    path = export_books_pdf(books, title='My Reading List')
    return f'Reading list exported to: {path}'

# Add to the return list:
return [save_book, remove_book, view_reading_list,
        count_reading_list, export_reading_list_pdf]
```

Then call agent_mgr.build() to rebuild all three graphs with the new tool. The Orchestrator automatically routes 'export my reading list as a PDF' to CuratorWorker, which now has the export tool available. Total change: ~12 lines, one file. This is exactly why the worker pattern exists ? extending one domain requires touching only that worker.

- Files changed: agents/curator_agent.py only
- Lines added: ~12
- Files NOT changed: agent.py, agents/search_agent.py, app.py, exporter.py

- Rebuild required: agent_mgr.build() Iqra Digital Library v2 | System Explainer & Scenario Q/A
(called automatically on next AI connect)